

National University of Computer and Emerging Sciences



Lab Manual 11 Artificial Intelligence

Course Instructor	Ms. Umm-e-Ammarah
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Spring 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Lab Manual 11	3
Objectives:.....	3
Activities:	3
Activity 1 (Introduction to CNNs):.....	3
1. Introduction to CNNs	3
Input Layer	3
Convolution Layer	4
Activation Function (ReLU)	4
Pooling Layer	4
Flatten Layer	5
Fully Connected (Dense) Layer	5
2. Implementing CNN using Python	5
Activity 2 (Problem Statements / Tasks):.....	7
1. CNN Architecture Understanding & Modification	7
2. CNN Implementation on MNIST	8
3. Effect of Pooling on Feature Maps	8
4. CNN Performance Improvement	9
5. CNN Layer-wise Output Interpretation	9

Lab Manual 11

Objectives:

- ✓ Understand the working of Convolutional Neural Networks (CNNs).
- ✓ Identify and explain different CNN layers (Convolution, Pooling, Fully Connected).
- ✓ Implement a CNN model using Python and deep learning libraries.
- ✓ Train and evaluate CNN models on image datasets.
- ✓ Interpret results using accuracy and loss curves.

Activities:

Activity 1 (Introduction to CNNs):

1. Introduction to CNNs

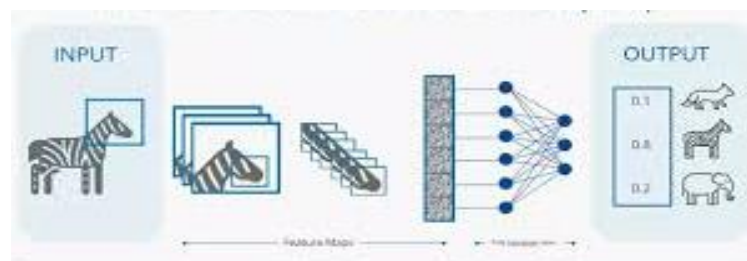
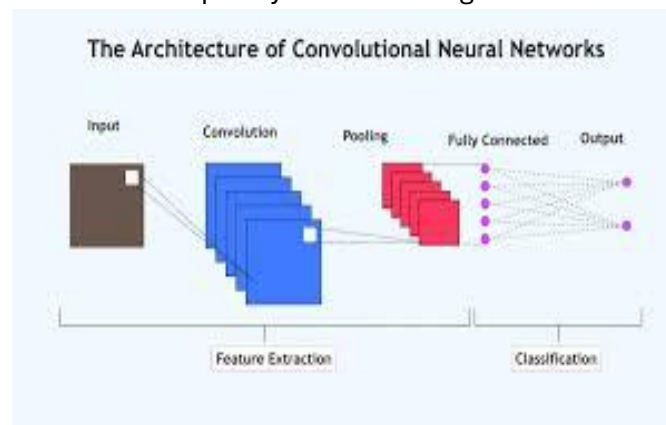
A Convolutional Neural Network (CNN) is a type of deep learning model primarily used for processing structured grid data such as images. CNNs automatically learn spatial hierarchies of features using convolution operations.

Unlike traditional neural networks, CNNs require less preprocessing and are highly effective in image classification, object detection, and pattern recognition tasks.

Images contain spatial information (height, width, channels). CNNs

- Preserve spatial relationships
- Require fewer parameters than fully connected networks
- Automatically extract features (edges, textures, shapes)

A typical CNN consists of multiple layers stacked together.



Input Layer

The input layer defines the shape of the data entering the network. For image data, this includes height, width, and number of channels.

Grayscale images have 1 channel, while RGB images have 3 channels.

```
tf.keras.Input(shape=(28, 28, 1))
```

shape=(28,28,1) means 28 = height, 28 = width, 1 = grayscale channel

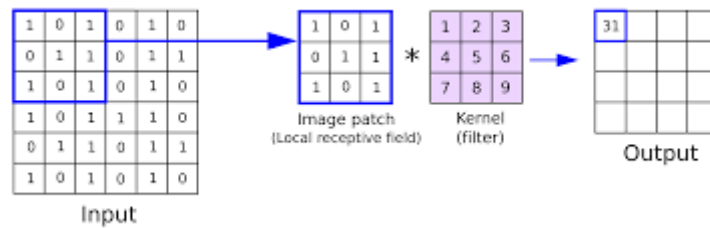
Convolution Layer

The convolution layer applies filters (kernels) to extract features from input images.

Mathematically:

$$\text{Feature Map} = \text{Input} \times \text{Kernel}$$

where: Input = Image matrix, Kernel = Filter matrix



```
tf.keras.layers.Conv2D(32, (3,3), activation='relu')
```

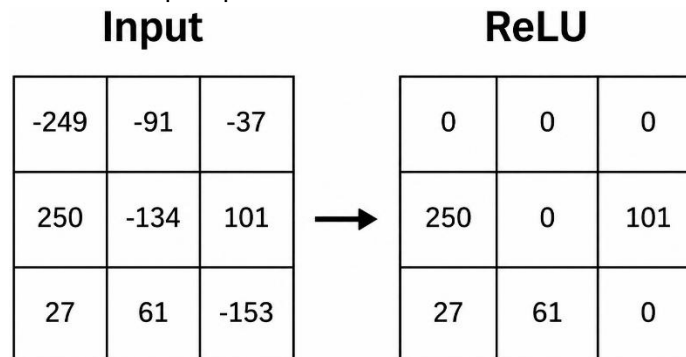
- 32 (filters): number of feature detectors
- (3,3): size of filter (kernel)
- activation='relu': introduces non-linearity

Activation Function (ReLU)

ReLU (Rectified Linear Unit) introduces non-linearity:

$$f(x) = \max(0, x)$$

It helps the network learn complex patterns.



Pooling Layer

Pooling reduces spatial dimensions and computation. Types:

- Max Pooling (most common)
- Average Pooling



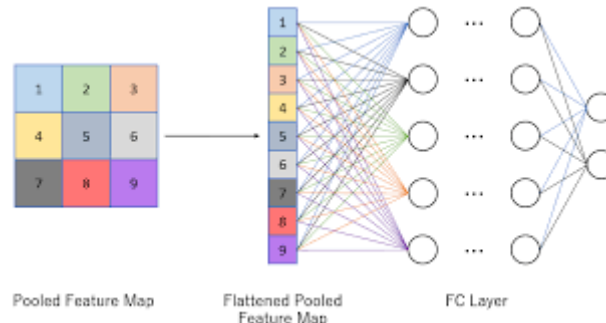
```
tf.keras.layers.MaxPooling2D((2,2))
```

pool_size: Size of pooling window (2x2 reduces dimensions by half)

Flatten Layer

Before passing data into dense layers, the 2D feature maps must be converted into a 1D vector.

```
tf.keras.layers.Flatten()
```



Fully Connected (Dense) Layer

Dense layers learn complex relationships between extracted features and output labels. The final layer usually uses softmax activation for multi-class classification.

```
tf.keras.layers.Dense(64, activation='relu')  
tf.keras.layers.Dense(10, activation='softmax')
```

2. Implementing CNN using Python

We will use a sample dataset such as handwritten digits (MNIST).

First, the dataset (MNIST) is loaded, which contains grayscale images of handwritten digits. The pixel values are normalized by dividing by 255 so that all values lie between 0 and 1, improving training efficiency. The images are reshaped into a 4D format (samples, height, width, channels) because CNNs expect channel information.

```
import tensorflow as tf  
from tensorflow.keras import datasets  
  
# Load dataset  
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()  
  
# Normalize data  
x_train = x_train / 255.0  
x_test = x_test / 255.0  
  
# Reshape for CNN  
x_train = x_train.reshape(-1, 28, 28, 1)  
x_test = x_test.reshape(-1, 28, 28, 1)
```

Next, a Sequential model is created, which allows stacking layers one after another. The first convolution layer applies 32 filters of size 3x3 to extract basic features from the image. This is followed by a max pooling layer that reduces the spatial dimensions, making computation efficient.

A second convolution layer with 64 filters is added to capture more complex patterns, followed again by pooling. The output is then flattened into a one-dimensional vector so it can be fed into dense layers.

A dense layer with 64 neurons learns high-level patterns, and the final dense layer with 10 neurons (one for each digit class) uses softmax activation to output probabilities.

```
import tensorflow as tf

model = tf.keras.Sequential([

    # Input Layer
    tf.keras.Input(shape=(28, 28, 1)),

    # Convolution + Pooling
    tf.keras.layers.Conv2D(32, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),

    # Second Convolution + Pooling
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),

    # Flatten
    tf.keras.layers.Flatten(),

    # Fully Connected Layers
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax') # Output Layer

])

model.summary()
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 64)	102,464
dense_3 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)

Trainable params: 121,930 (476.29 KB)

Non-trainable params: 0 (0.00 B)

The model is compiled using the Adam optimizer and a suitable loss function for classification. It is then trained for multiple epochs, during which it learns patterns in the data.

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy', # for multiclass
              metrics=['accuracy'])

history = model.fit(x_train, y_train, epochs=5,
                  validation_split=0.2)
```

Finally, the model is evaluated on test data to measure its accuracy (you can find other evaluation metrics as well in the same manner we covered under ANN (Tensorflow-Lab 10)).

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_acc)
```

Visualization of Training

```
import matplotlib.pyplot as plt

# Accuracy Plot
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title("Accuracy Curve")
plt.show()

# Loss Plot
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Loss Curve")
plt.show()
```

Activity 2 (Problem Statements / Tasks):

1. CNN Architecture Understanding & Modification

You are given a CNN model:

```
model = tf.keras.Sequential([
    tf.keras.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(32, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

Tasks:

- Display model summary and identify output shape after each layer
- Calculate:
 - Output size after first Conv2D
 - Output size after first MaxPooling
- Modify the model:
 - Add one more Conv2D layer (128 filters)
 - Add Dropout (0.3) before final dense layer `tf.keras.layers.Dropout(0.3),`
- Explain:
 - Why pooling is used
 - Why ReLU is preferred

2. CNN Implementation on MNIST

Use MNIST dataset.

Tasks:

- Load and preprocess data:
 - Normalize
 - Reshape to (28,28,1)
- Build CNN:
 - Conv → Pool → Conv → Pool → Flatten → Dense → Output
- Train model for 5 epochs
- Display:
 - Test accuracy
 - Training vs validation accuracy graph
- Modify:
 - Change optimizer from Adam → SGD `model_sgd.compile(optimizer='sgd', loss='sparse_categorical_crossentropy', metrics=['accuracy'])`
- Compare:
 - Accuracy difference (Adam vs SGD)

Analysis:

- Which optimizer performed better and why?

3. Effect of Pooling on Feature Maps

You are given a matrix (feature map):

```
[[4, 0, -2, 4],  
 [3, 1, 0, 2],  
 [1, 0, 1, 1],  
 [4, 6, 5, 1]]
```

Tasks:

- Apply:
 - ReLU activation
- Apply:
 - Max Pooling (2×2, stride=2)
 - Average Pooling (2×2)
- Compare outputs:
 - Max vs Average pooling

- Explain:
 - Which preserves strongest features?
 - Which smooths information?

4. *CNN Performance Improvement*

You are given a CNN model trained on MNIST but showing:

- High training accuracy
- Low validation accuracy

Tasks:

- Identify problem → Overfitting
- **Modify model to fix it:**
 - **Add Dropout**
 - **Add Batch Normalization (optional if known)**
- Train again and compare:
 - Training accuracy
 - Validation accuracy
- Plot:
 - Loss curve (before vs after)

Analysis:

- Why does dropout help?
- What happens if dropout is too high?

5. *CNN Layer-wise Output Interpretation*

Given CNN flow:

Input → Conv → ReLU → Pool → Flatten → Dense

Tasks:

- Explain role of each layer in your own words
- Given:
 - Input size = $(28 \times 28 \times 1)$
 - Conv layer: 32 filters, (3×3) , stride=1
 - Pooling: (2×2)
- Compute:
 - Output after Conv
 - Output after Pooling
 - Size after Flatten
- Explain:
 - Why Flatten is required
 - What happens if Flatten is skipped